Lecture 11 K-Means, unsupervised GMM, EM

Recap: Unsupervised Learning

- A dataset without labels
  - Learn interesting structures within the data
- Clusters of related datapoints
  - E.g., customer segmentation.
- Outliers, i.e., particularly unusual or interesting datapoints.
- Denoised signals.
- Dimensionality reduction

Recap: K-Means

- The performance of K-Means is sensitive to the scale of the variables
- K-means performs the best when the variables are uncorrelated (i.e., they form circular clusters) and have the same variances across clusters

k-Means Clustering

Set number of clusters

- Select k number of clusters

Initialize

- Randomly select k centroid locations

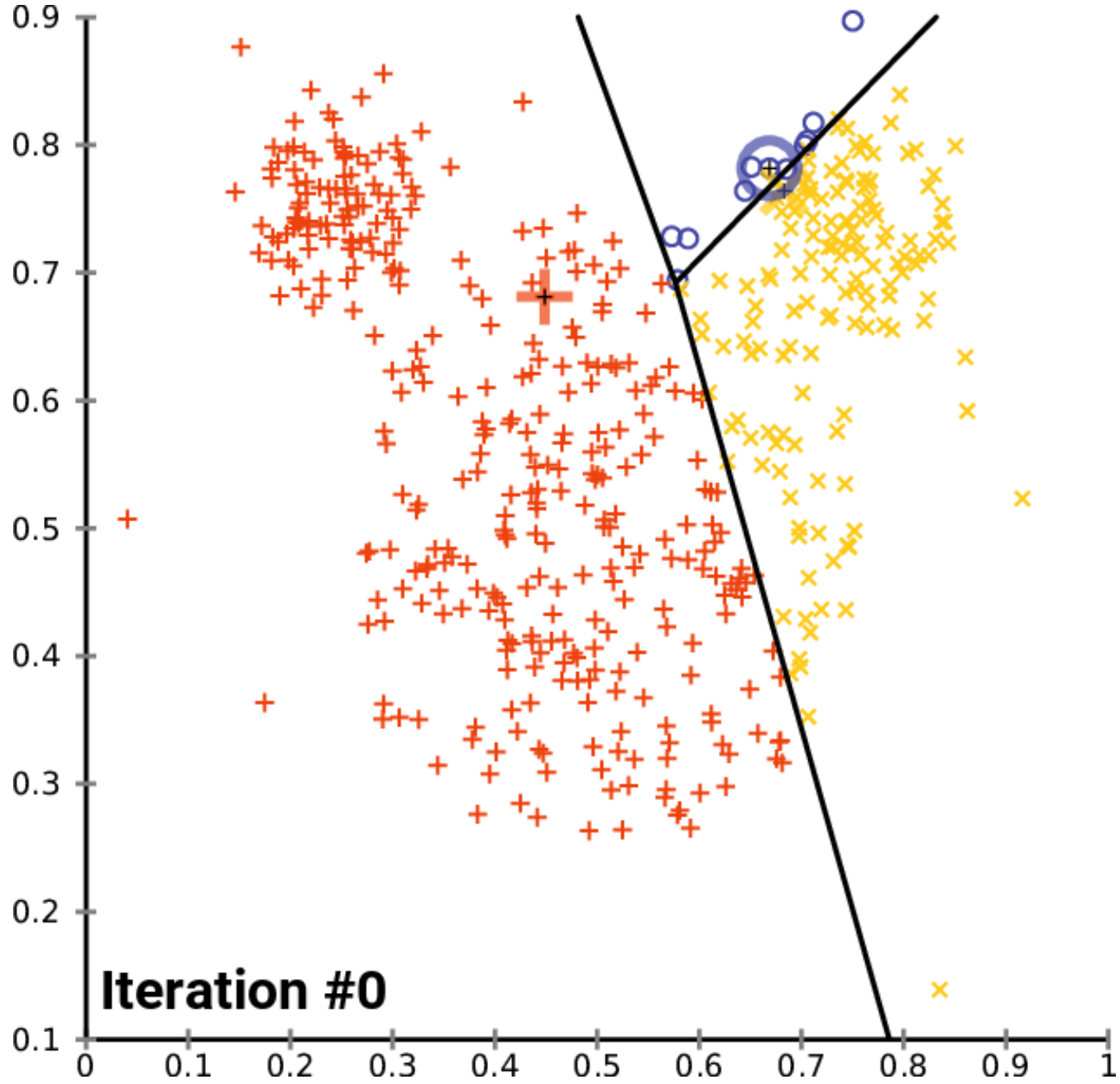
Iterate

- Repeat until convergence
  - Update $f(x)$ such that
    $$f\big(x(i)\big) = \text{argmin}_k \big|\big|x(i) - c_k\big|\big|_2$$
  - is the cluster of the closest centroid to $x(i)$.
  - Set each c_k to be the center (the average vector) of the its cluster $\{x(i)|f(x(i))=k\}$

Terminate

- Terminate when no observations get reassigned

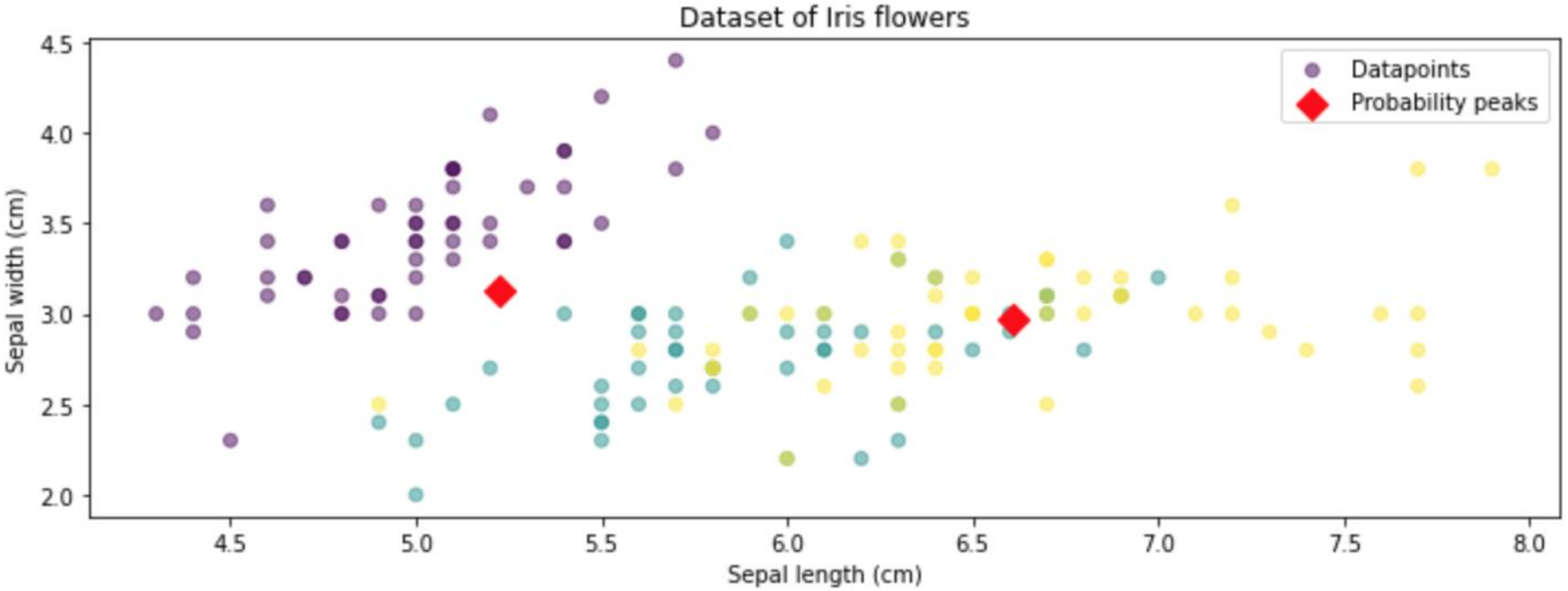
Illustrated visually (from Wikipedia):



k-Means Clustering

Selecting the Number of Clusters k

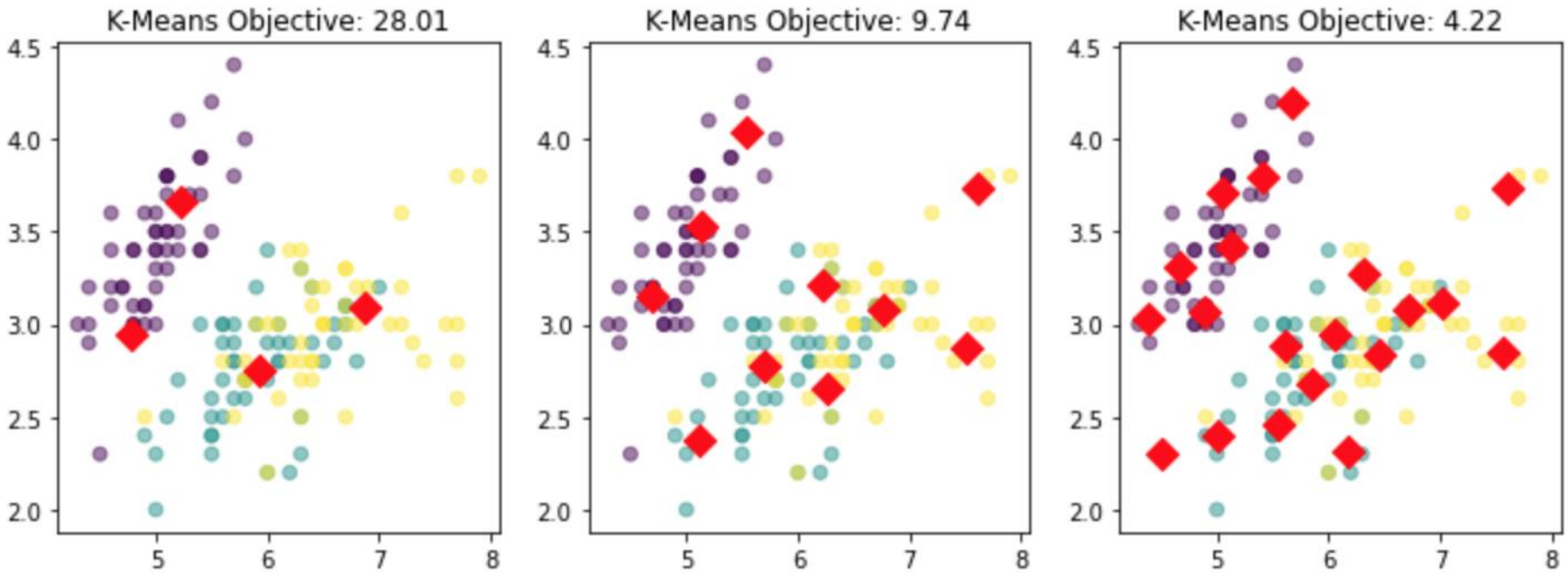
Selecting the Number of Clusters k



Selecting the Number of Clusters k

```
Ks = [4, 10, 20]
f, axes = plt.subplots(1,3)
for k, ax in zip(Ks, axes):
    model = cluster.KMeans(n_clusters=k)
    model.fit(iris.data[:,[0,1]])
    ax.scatter(iris.data[:,0], iris.data[:,1], c=iris.target, alpha=0.5)
    ax.scatter(model.cluster_centers_[:,0],
               model.cluster_centers_[:,1], marker='D', c='r', s=100)
    score = model.score(iris.data[:,[0,1]])
    ax.set_title('K-Means Objective: %.2f' % -score)
```

Selecting the Number of Clusters k



Generalization in Unsupervised Learning

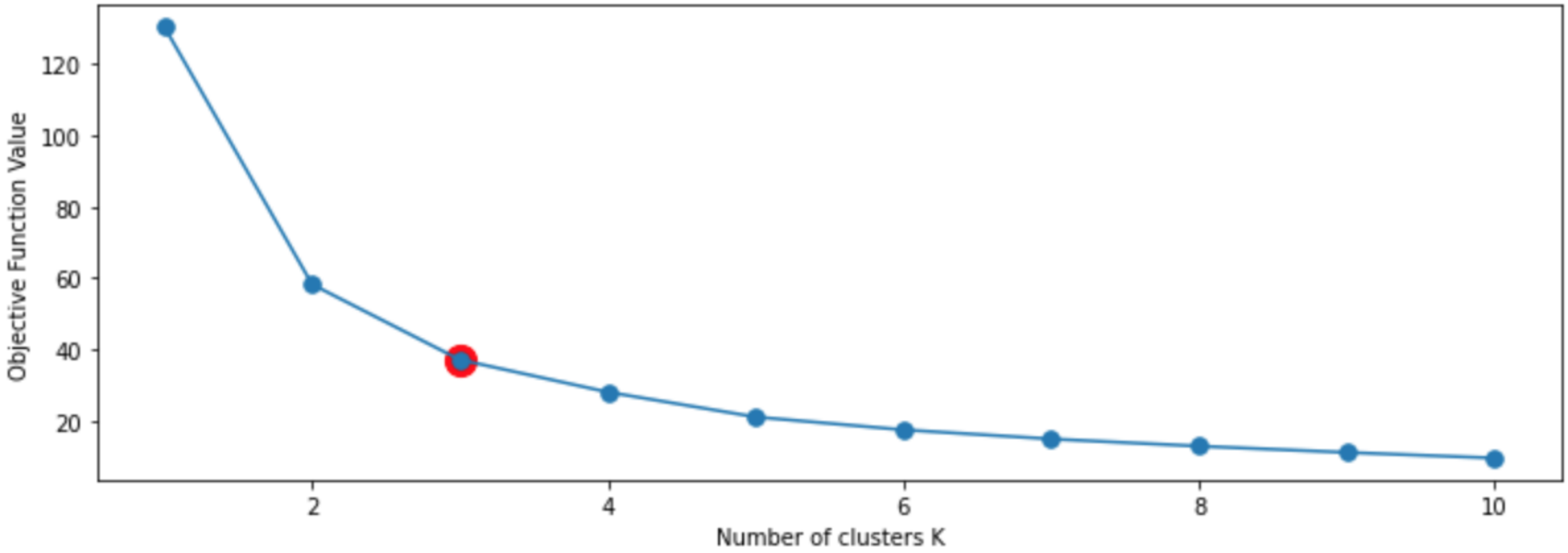
- **Generalization** measures the performance of predictive models on
  - New, unseen data that is distinct from the training set.
- In **unsupervised learning**, think of **data distribution** as
  - The sum of two distinct components $P = F + E$
  - F : A signal component
    - Hidden clusters, speech, low-dimensional data space, etc.
  - E : A random noise component

Generalization in Unsupervised Learning

- In **unsupervised learning**, think of **data distribution** as
  - The sum of two distinct components $P = F + E$
  - F : A signal component
  - E : A random noise component
- Underfit: does not learn signal F
- Generalizes: learns signal F
- Overfit: mistaken E for F

Elbow Method

- Plot the objective function as a function of the hyper-parameter K
- The "elbow" of the curve happens when its rate of decrease substantially slows down.
- The "elbow' is a good guess for the hyperparameter.



Detecting Overfitting and Underfitting

- More difficult to quantify than supervised learning
- Performance may depend on our intuition and require human evaluation
- If we know the true labels, we can measure the accuracy of the clustering
- If not, and the model is probabilistic
  - can detect overfitting without labels by comparing the log-likelihood between the training set and a holdout set

Are the following questions on unsupervised learning correct?

0

Overfitting in unsupervised learning can be reduced by reducing the model complexity (e.g., reduce K in K-Means)

0

Overfitting in unsupervised learning can be reduced by using a probabilistic model and penalizing the number of clusters in the objective function.

0

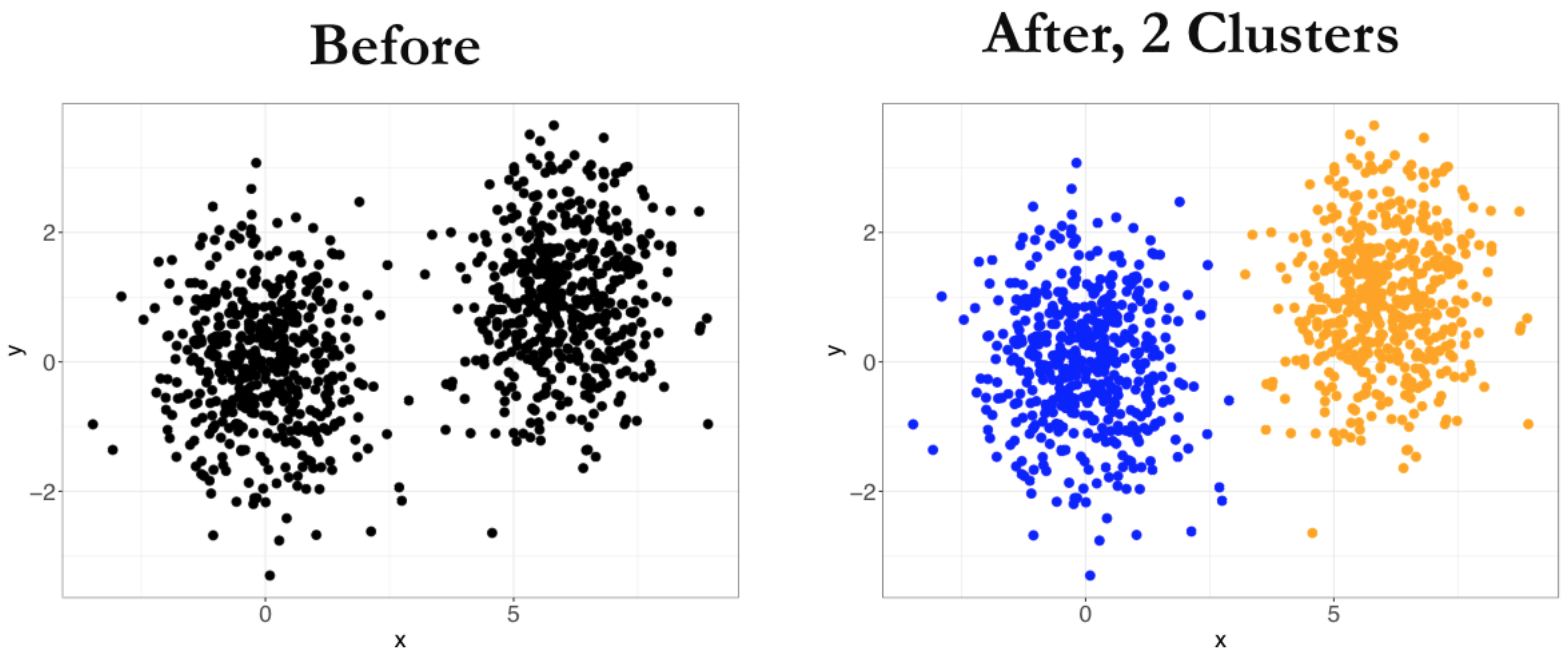
Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Gaussian Mixture Models

Overcoming deficiencies in k-Means

k-Means and Correlation

- Here's a scatterplot that is particularly well suited to k-Means clustering:



k-Means and Correlation

Suppose I create a new data set with M+1 columns where:

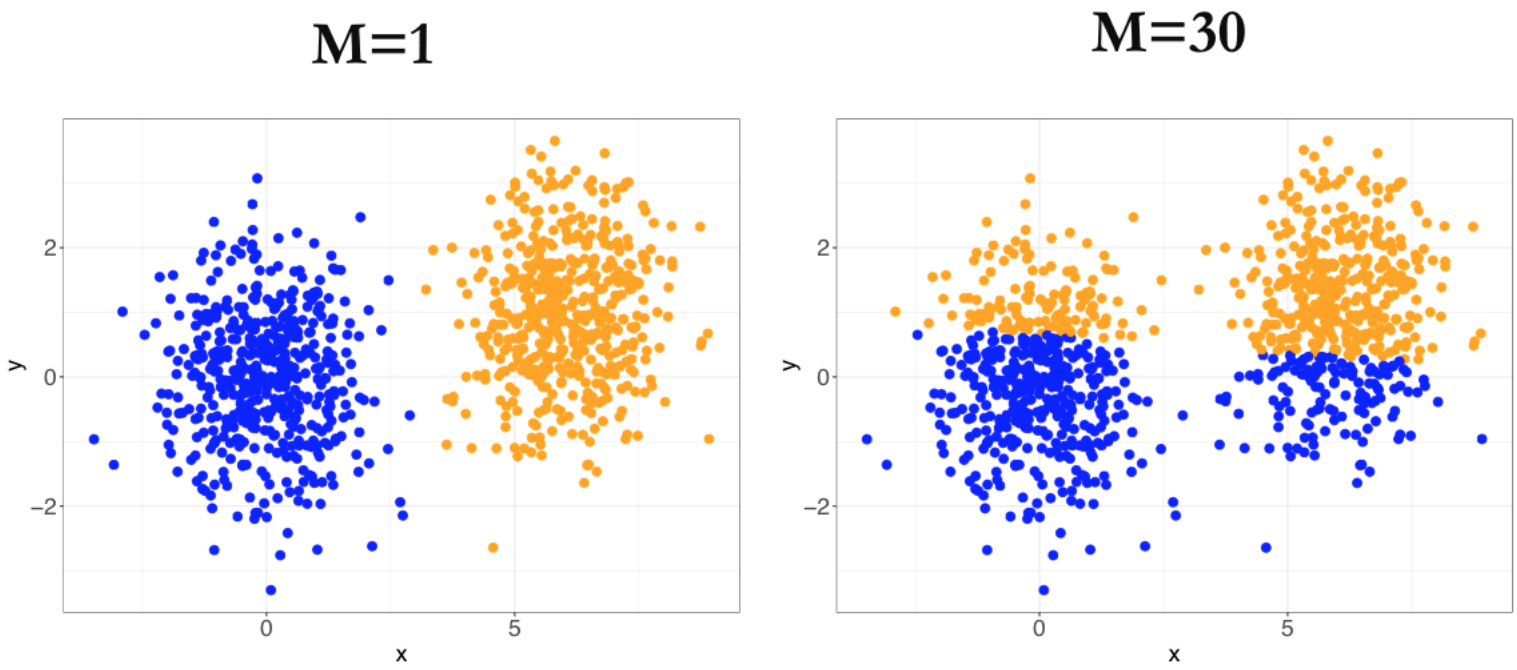
- x is the 1st column
- y is repeated in the remaining M columns

After doing this, I run k-Means as before on this new data set

- Distance between two points a and b is:

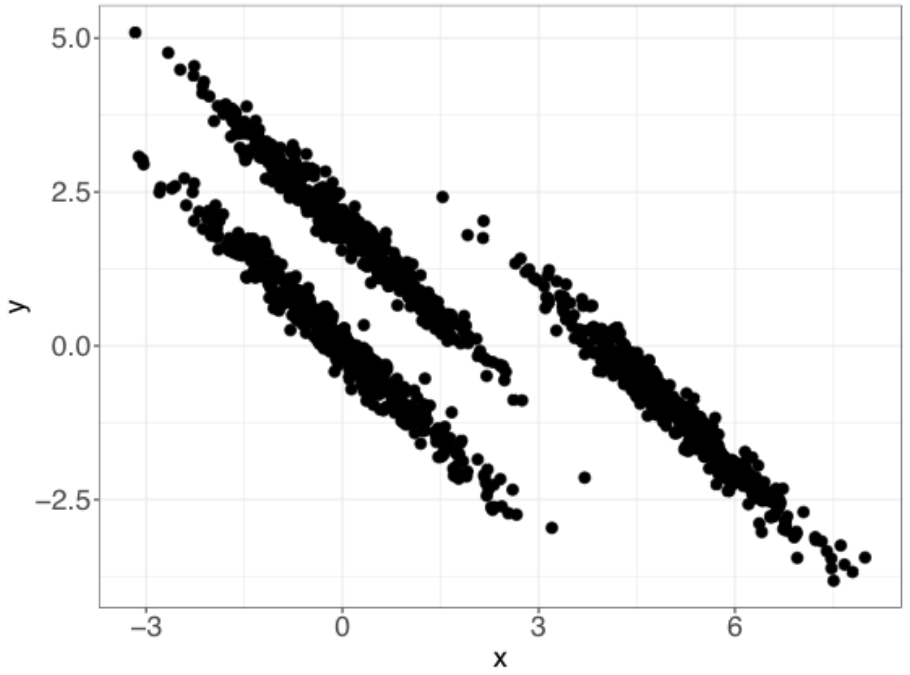
- j is the jth column $\sqrt{\sum_{j=1}^{M+1}(a_j - b_j)^2}$

k-Means and Correlation



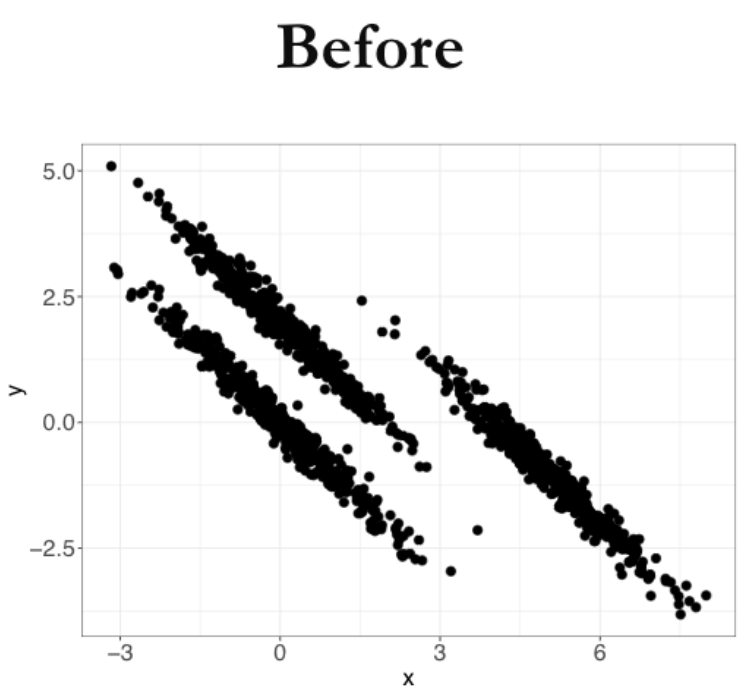
k-Means and Correlation

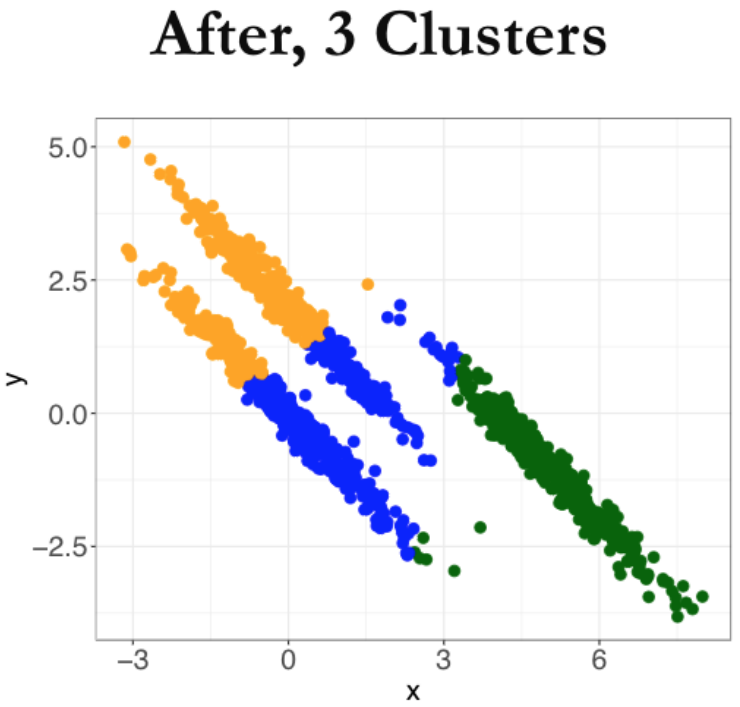
- How many clusters do you think there are?
- What shape are the clusters?
- Is there strong or weak correlation between the variables within each cluster?



k-Means and Correlation

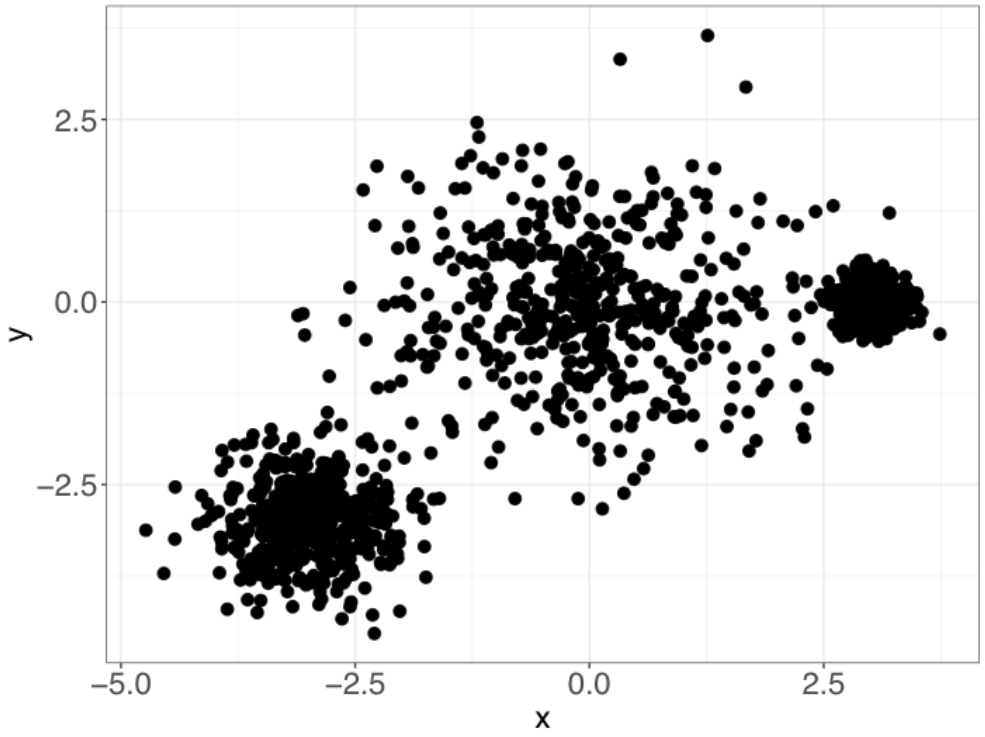
- Here are the results from k-Means. Rough times.





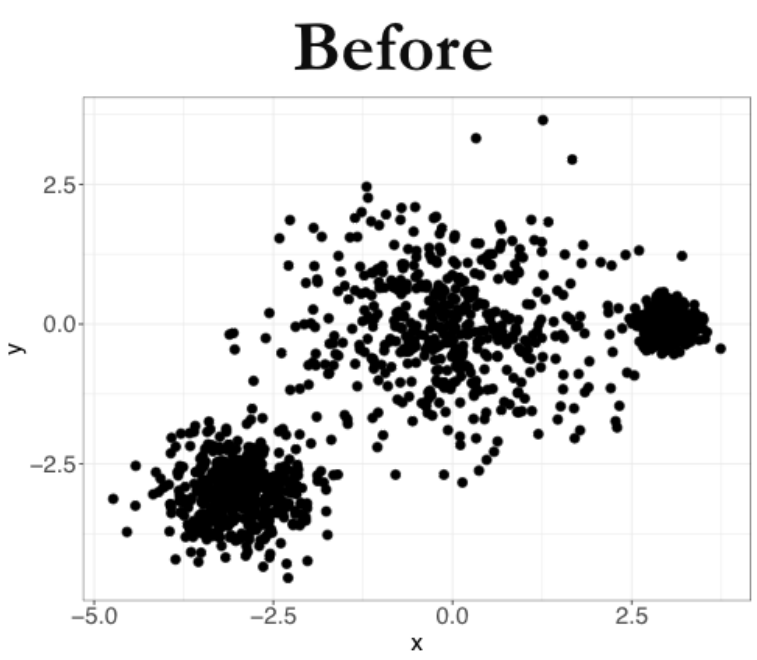
Another Example

- How many clusters do you think there are?
- What shape are the clusters?
- Is there strong or weak correlation between the variables within each cluster?



k-Means and Unequal Variances

- Here are the results from k-Means. OK for the most part, but we can spot some rough spots





k-Means and Correlation

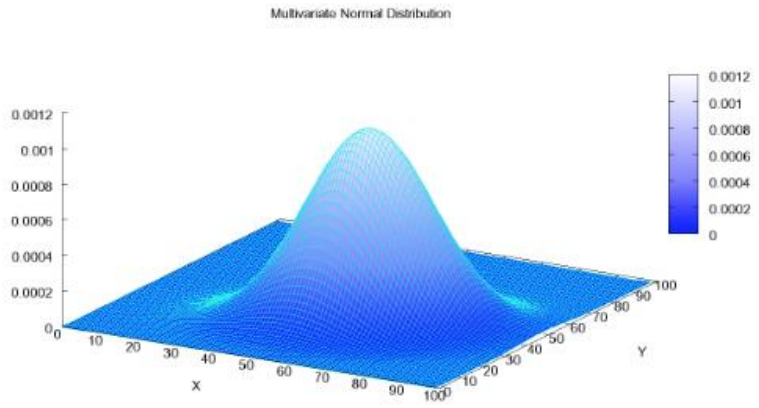
- k-Means does best when the clusters are
  1. Circular/spherical shaped (uncorrelated variables); and
  2. Have the same variances across clusters
- Unfortunately, oftentimes clusters are
  1. Best described by ellipsoids (correlated variables); and/or
  2. Have different variances
- In these situations, k-Means will often not perform well!

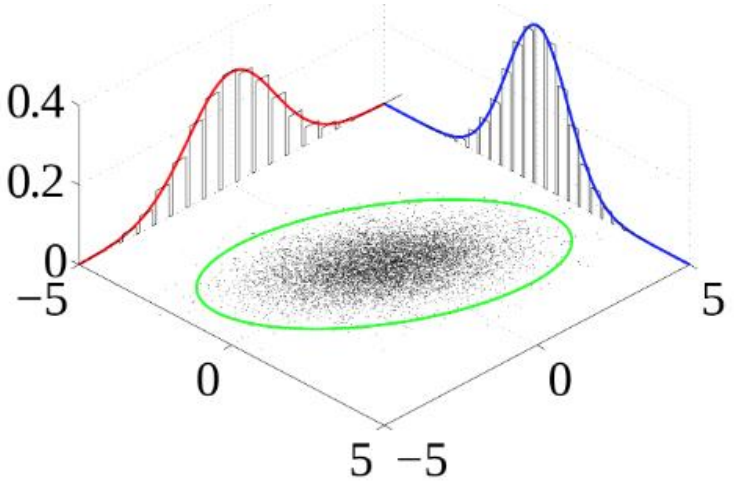
k-Means Other Limitations

- k-Means clustering can get stuck at local minima
- Measuring clustering quality is hard and relies on heuristics
- Clustering assignment is binary and does not estimate confidence

Gaussian Mixture Models

- **Gaussian Mixture Models (GMMs)** provide an improved algorithm for clustering when clusters are elliptical and/or may have different variances
- Clustering is based upon the multivariate analogue of the normal, or **Gaussian**, distribution





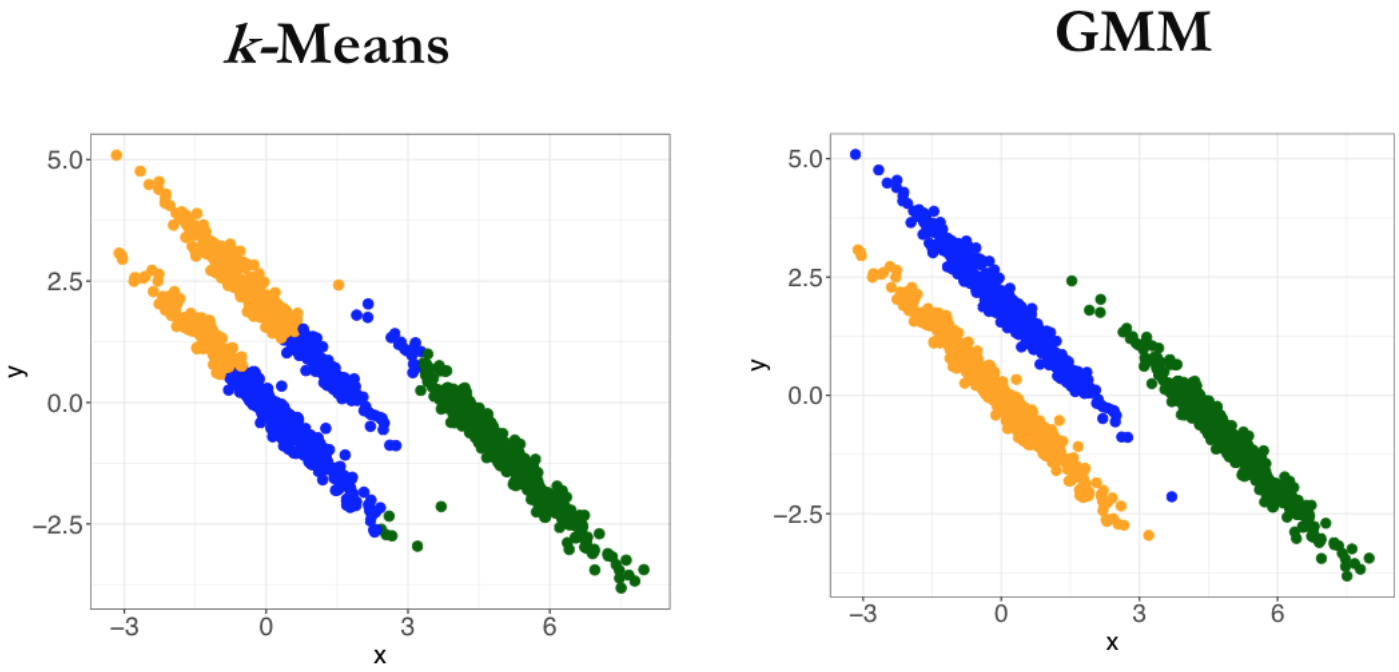
Gaussian Mixture Models

- Assume data is generated according to K distinct multivariate normal distributions
  - With **unknown** means, variances, and correlation structure
  - We don't know which data point came from which distribution
- We can find out the values for the means, variances, and correlations that would make the data the most likely to be observed
  - Using the Expectation-Maximization (EM) algorithm

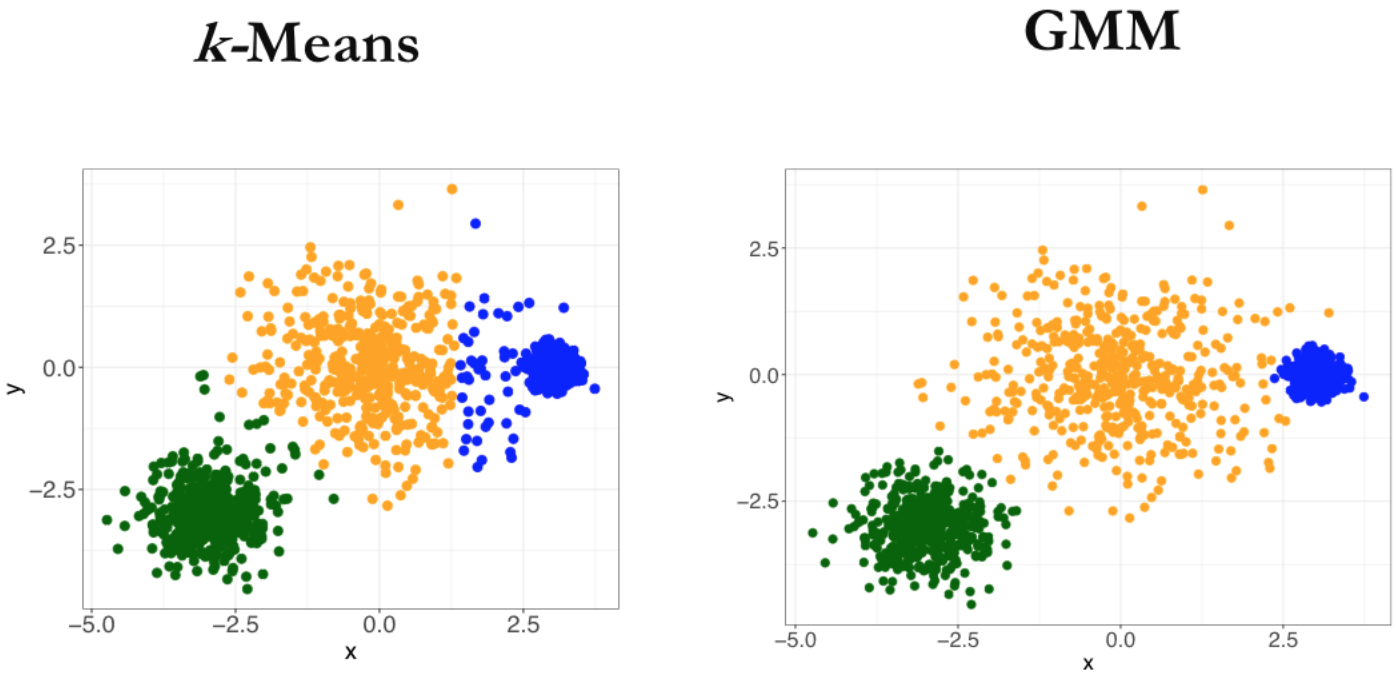
Gaussian Mixture Models

- Once we have the most likely K clusters, we can calculate the **probability** that an individual is from a given cluster
- We assign an individual to the cluster with the highest probability

Comparing GMMs and k-Means



Comparing GMMs and k-Means

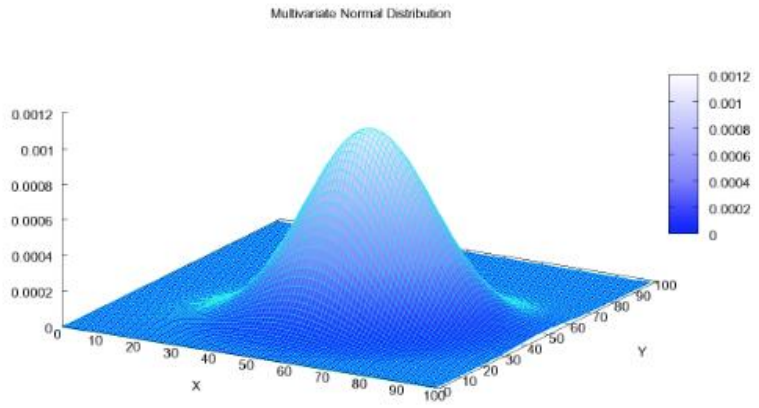


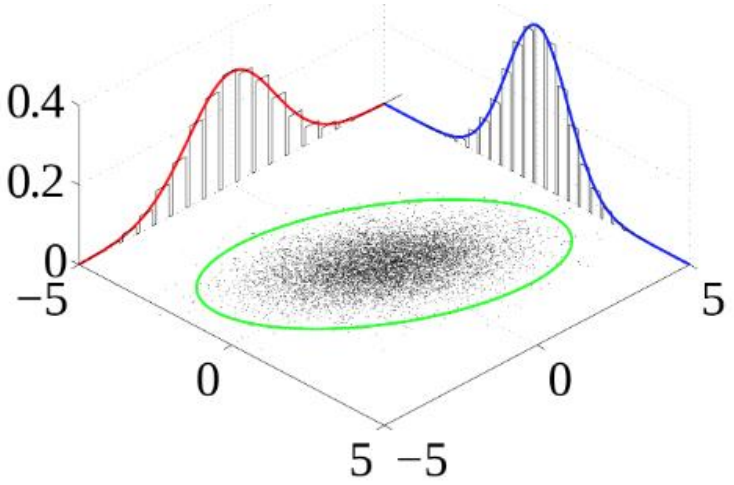
Unsupervised Probabilistic Model

- Probability distribution $P_\theta(x)$: $\chi \rightarrow [0,1]$.
- θ: model parameter
  - e.g., Bernoulli if binary observations
  - e.g., Gaussian when observations are continuous
- Density estimation:
  - Learn $P_\theta(x)$ from data to approximate true data generating distribution P_{data}

Gaussian Mixture Models

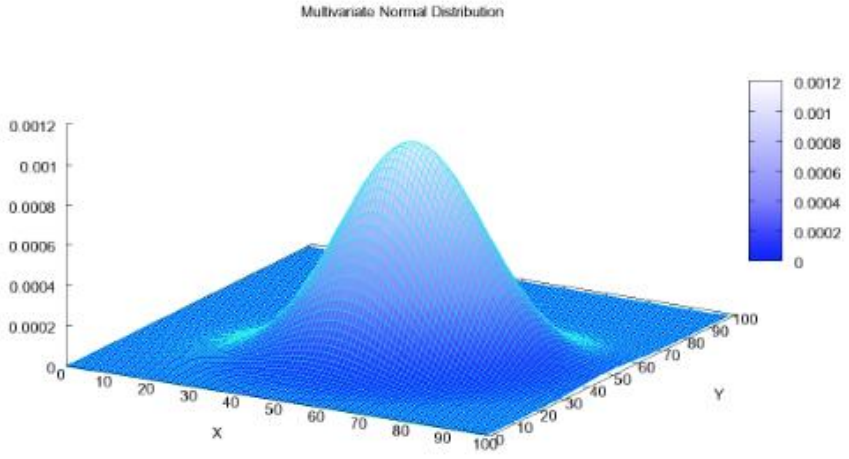
- **Gaussian Mixture Models (GMMs)** provide an improved algorithm for clustering when clusters are elliptical and/or may have different variances
- Clustering is based upon the multivariate analogue of the normal, or **Gaussian**, distribution

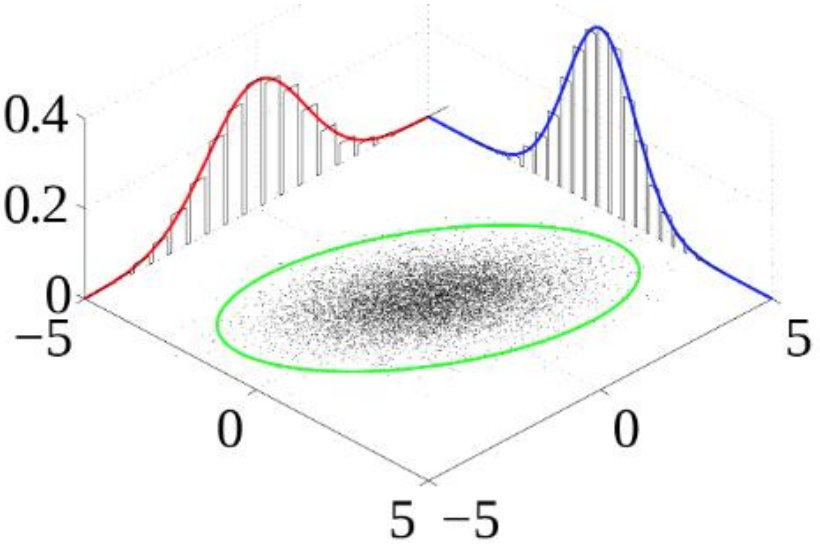




Gaussian Mixture Models

- Multivariate Gaussian Distribution $N(\boldsymbol{u}, \boldsymbol{\Sigma})$
  - $\boldsymbol{u}$: vector of length d
    - u_1: age; u_2: height …
  - $\boldsymbol{\Sigma}$: d by d matrix denote the covariance

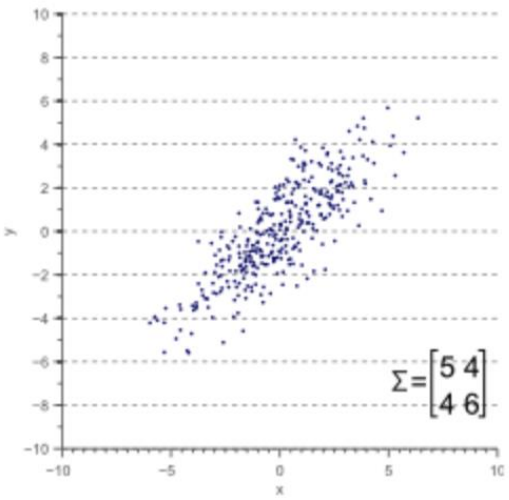


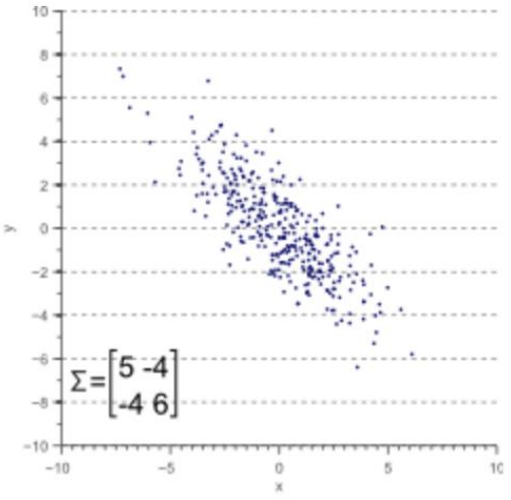


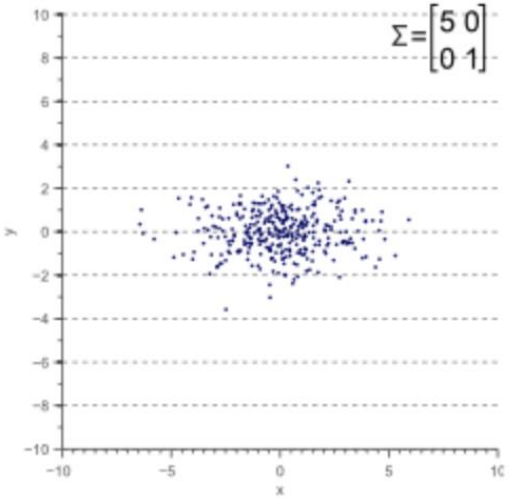
Covariance Matrix

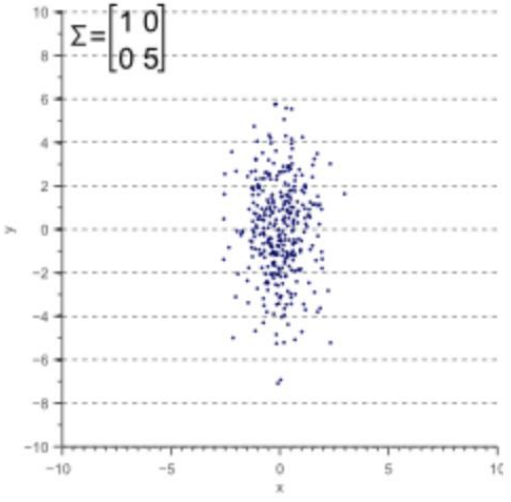
$$\operatorname{cov}(X, Y) = \operatorname{E}\left[(X - \operatorname{E}[X])(Y - \operatorname{E}[Y])\right]$$

$$\begin{array}{c} \\ x \\ y \end{array} \begin{array}{c} \begin{array}{cc} x & y \end{array} \\ \begin{bmatrix} var(x) & cov(x,y) \\ cov(x,y) & var(y) \end{bmatrix} \end{array}$$







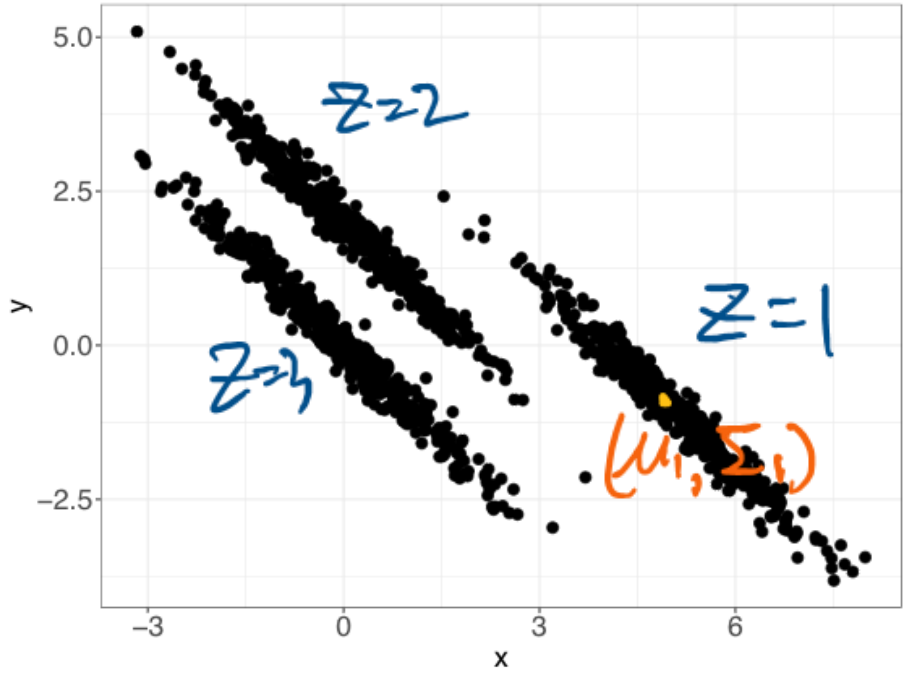


Gaussian Mixture Models Notation

- Let z be the index for clusters
- For each cluster z, we model the data as a multivariate Gaussian distribution

$$P(x|z=k) = N(\boldsymbol{u_k}, \boldsymbol{\Sigma_k})$$

$$x, \boldsymbol{u}_k \in \mathbb{R}^d, \boldsymbol{\Sigma_k} \in \mathbb{R}^{d \times d}$$



how to learn (μ_1, Σ_1)?

Gaussian Mixture Models Notation

- For each cluster z, we model the data

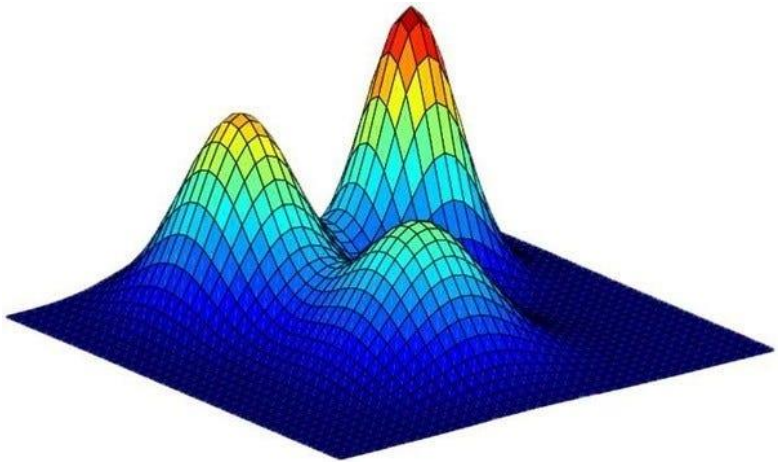
$$P(x|z = k) = N(\boldsymbol{u_k}, \boldsymbol{\Sigma_k})$$

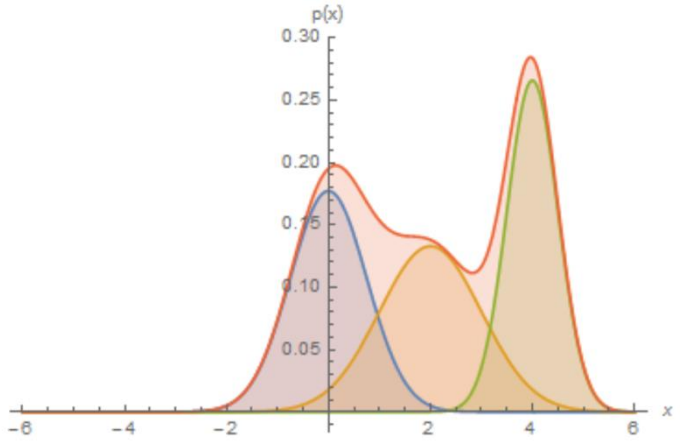
$$x, \boldsymbol{u}_k \in \mathbb{R}^d, \boldsymbol{\Sigma_k} \in \mathbb{R}^{d \times d}$$

- Gaussian Mixture

$$P_\theta(x, z) = P_\theta(x|z)P_\theta\,(z)$$

  - where $P_\theta(z = k) = \phi_k$
  - The parameters θ are the $\mathbf{u}_k, \boldsymbol{\Sigma}_k, \phi_k$ for all k=1,2,…,K





Unsupervised Gaussian Mixture Models Setup

- K distinct multivariate Gaussian distributions generating data: $P(x|z = k)$ = $N(\boldsymbol{u_k}, \boldsymbol{\Sigma_k})$
  - Unknown means, variances, and correlation structures: need to learn$(\boldsymbol{u_k}, \boldsymbol{\Sigma_k})$
- Unlabeled data -> unknown cluster mixture: $P_\theta(z = k) = \phi_k$

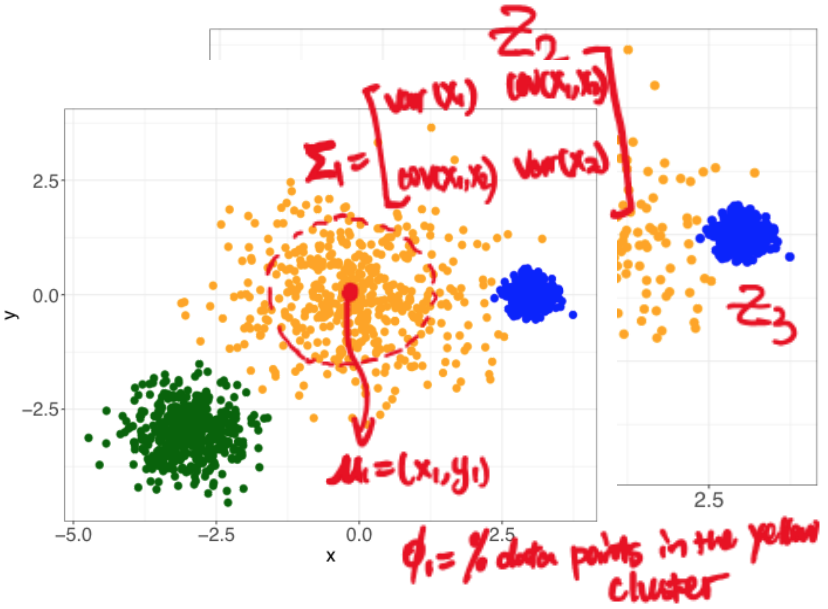
Gaussian Mixture Models Notation

- Let z be the index for clusters

(colors)

GMM

- For each cluster z, we model the data as a multivariate Gaussian distribution



  - $P(x|z = k) = N(\boldsymbol{u_k}, \boldsymbol{\Sigma_k})$
  - $x, \boldsymbol{u}_k \in \mathbb{R}^d, \boldsymbol{\Sigma_k} \in \mathbb{R}^{d \times d}$
- Gaussian Mixture

$$P_\theta(x, z = k) = P_\theta(x|z = k)P_\theta\,(z = k)$$

  - where $P_\theta(z = k) = \phi_k$
  - The parameters θ are the $\mathbf{u}_k, \boldsymbol{\Sigma}_k, \phi_k$ for all k=1,2,…,K

Unsupervised GMM Learning

Given dataset, find the values for the means, variances, correlations, and cluster proportion that would make the data the most likely to be observed

- $P_\theta(x, z = k) = P_\theta(x|z = k)P_\theta\,(z = k)$
- Using the **Expectation-Maximization (EM) algorithm to learn $\boldsymbol{u_k}, \boldsymbol{\Sigma_k}, \phi_k$**
- Model postulates that our observed data is comprised of K clusters with proportions specified by $\phi_1, \phi_2, \ldots, \phi_K$
- Points within each cluster follow a Normal distribution $P(x|z = k)$ $N(\boldsymbol{u_k}, \boldsymbol{\Sigma_k})$

Unsupervised GMM Inference

Calculate the **probability** that a datapoint is from a given cluster; Assign datapoint to the cluster with the highest probability

- $z^{(i)}$ is the cluster label for data point $x^{(i)}$
- Cluster assignment probability can be obtained from $P(z^{(i)}|x^{(i)})$.
- Bayes rule
  - $P_\theta(x, z = k) = P_\theta(x|z = k)P_\theta\,(z = k)$

$$P_\theta(x) = \sum_{k=1}^{K} P_\theta(z = k)P_\theta(x|z = k)$$
$$= \phi_1 \mathcal{N}(x; \mu_1, \Sigma_1) + \ldots + \phi_K \mathcal{N}(x; \mu_K, \Sigma_K)$$

Warm-Up: Clustering with Known Cluster Assignments

- Suppose we have a dataset where the identity of each cluster is known

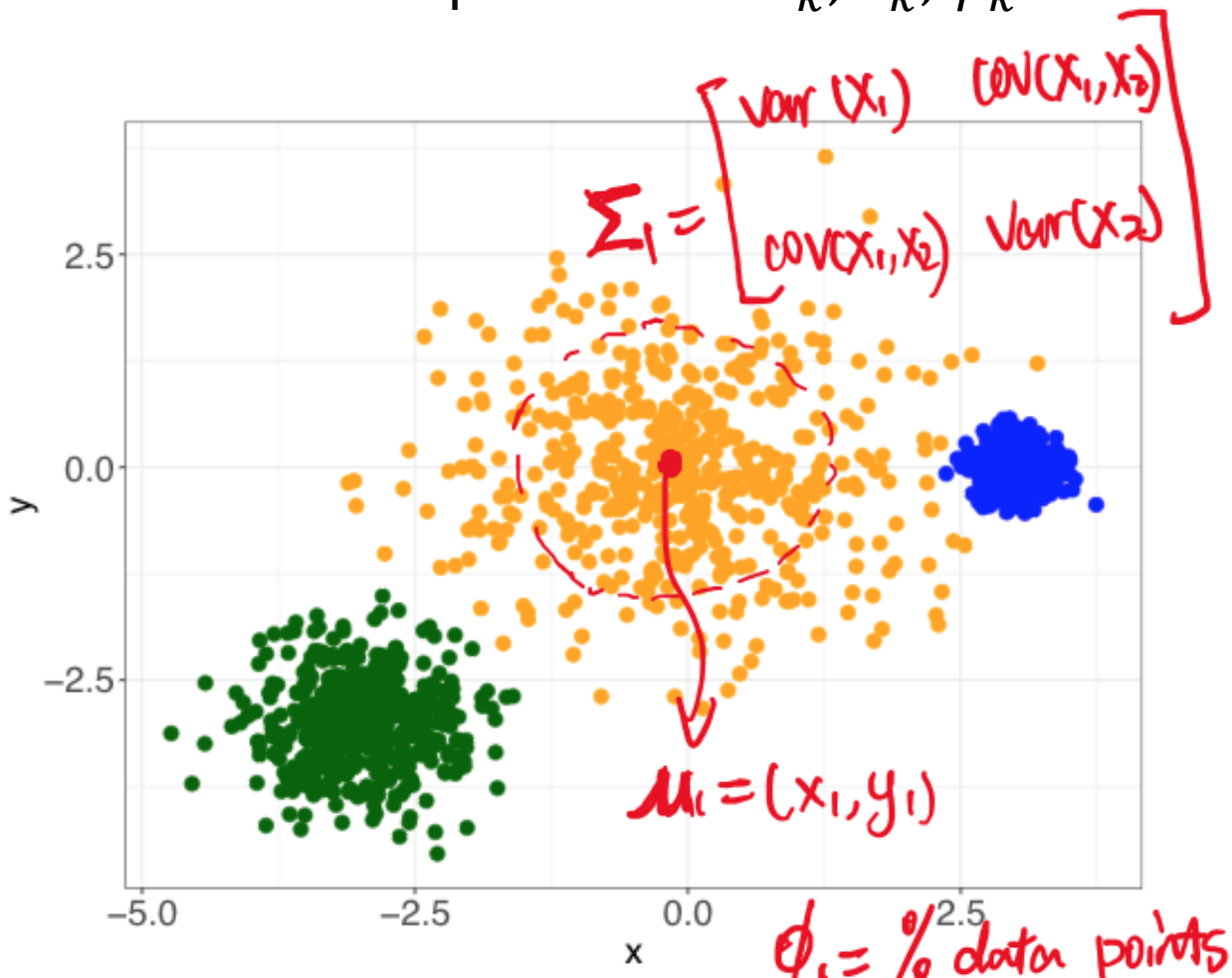
$$D = \{(x^{(1)}, z^{(1)}), (x^{(2)}, z^{(2)}), \ldots, (x^{(n)}, z^{(n)})\}$$

- $x^{(i)} \in \mathbb{R}$ are the datapoints we cluster
- $z^{(i)} \in \{1,2, \ldots, K\}$ are the cluster IDs
- We will fit a GMM, the μ_k will represent the cluster centroids.

- How do we do this?

GDA

- If we are provided with the clustering assignment as below, how do we learn the parameters $\mathbf{u}_k, \boldsymbol{\Sigma}_k, \phi_k$?



Gaussian Discriminative Analysis

$$D = \{(x^{(1)}, z^{(1)}), (x^{(2)}, z^{(2)}), \ldots, (x^{(n)}, z^{(n)})\}$$

- Fix cluster IDs for sample $x^{(i)}$, $z^{(i)} \in \{1,2, \ldots, K\}$
- Objective: $\max_{\theta} \frac{1}{n} \sum_{i=1}^{n} \log P_{\theta}(x^{(i)}, z^{(i)})$.
- Fit a GMM. Let μ_k represent the cluster centroids:

1. We fit the parameters ϕ_k to be % of each class k in D.
2. We fit the parameters μ_k, Σ_k to be empirical means and variances of each class k.
3. We can infer the cluster ID z of new points using $P(z \mid x)$.

From Labeled to Unlabeled Clustering

Learn GMM from unlabeled data:

- Jointly learn cluster parameters and labels:

1. We will define an objective that does not require cluster labels $z^{(i)}$.
2. We will define an optimizer that jointly infers both labels $z^{(i)}$ and cluster parameters μ_k, Σ_k.

Maximum Marginal Likelihood Learning

- Goal: maximize the total likelihood of the data

$$\frac{1}{n}\sum_{i=1}^{n}\log P_\theta(x^{(i)})$$

Obtain joint probability of $x^{(i)}$ and z

$$P_\theta\left(x^{(i)}\middle|z\right)$$

- From GDA, we kind of know how to write down $P_\theta\left(x^{(i)}\middle|z\right)$
- But get an extra z term when compared with objective

$$p\left(x^{(i)}|z^{(i)}=j;\mu,\Sigma\right)=\frac{1}{(2\pi)^{n/2}|\Sigma_j|^{1/2}}\exp\left(-\frac{1}{2}\left(x^{(i)}-\mu_j\right)^T\Sigma_j^{-1}\left(x^{(i)}-\mu_j\right)\right)$$

Maximum Marginal Likelihood Learning

- Goal: maximize the total likelihood of the data

$$\frac{1}{n}\sum_{i=1}^{n}\log P_\theta(x^{(i)})$$

Obtain joint probability of $x^{(i)}$ and z

$$P_\theta\left(x^{(i)}\middle|z\right)P_\theta(z) = P_\theta(x^{(i)}, z)$$

Marginalize over the cluster ID z:

$$\sum_{z=1}^{K} P_\theta(x^{(i)}, z) = P_\theta(x^{(i)})$$

Maximum Marginal Likelihood Learning

- Goal: maximize the total likelihood of the data by marginalizing over the cluster IDs:

$$\frac{1}{n}\sum_{i=1}^{n}\log P_\theta(x^{(i)}) = \frac{1}{n}\sum_{i=1}^{n}\log\sum_{z=1}^{K} P_\theta(x^{(i)}, z)$$

- This asks P_θ to assign a high probability to the training data in D.
  - z encodes the cluster id; we ask P_θ to be high for ***at least one*** z
- Need to use $P(x) = \sum_{z \in Z} P(x, z)$ to compute this probability because z is not observed.

Optimizing Marginal Likelihood

$$\max_{\theta} \frac{1}{n} \sum_{i=1}^{n} \log \sum_{z=1}^{K} P_\theta(x^{(i)}, z)$$

- No closed-form solutions because we cannot slip the sum and the log
  - The objective now has many local minima!
- The optimization would have been easier if $P_\theta(z)$ is known
- Unknown $P_\theta(z)$:
  - Creates dependency between the learned covariate distribution conditioned on the cluster ID (μ_k, Σ_k) and the cluster assignments as they now depend on model parameters
  - Can best compute $P_\theta\left(z|x^{(i)}\right)$ at each iteration!

Learn GMM from unlabeled data

Jointly learn cluster parameters and labels:

1. Objective marginalized over the cluster labels $z^{(i)}$.
2. Optimizer jointly infers both labels $z^{(i)}$ and cluster parameters μ_k, Σ_k.

$$\max_{\theta} \frac{1}{n} \sum_{i=1}^{n} \log \left(\sum_{z \in \mathcal{Z}} P_\theta(x^{(i)}, z) \right)$$

- The above objective is hard to solve, so we use the EM algorithm

Recovering Clusters from GMMs

Use Bayes rule to calculate the posterior probability

- Given a trained GMM model

$$P_\theta(x, z) = P_\theta(x|z)P_\theta(z)$$

- Compute the probability of a point x belonging to class k

$$P_\theta(z = k \mid x) = \frac{P_\theta(z = k, x)}{P_\theta(x)} = \frac{P_\theta(x|z = k)P_\theta(z = k)}{\sum_{l=1}^{K} P_\theta(x|z = l)P_\theta(z = l)}$$

- Probabilistic cluster assignment

Are the following questions on unsupervised GMM correct?

0

In GMM clustering, we start by randomly initializing the centers and covariances of the clusters; we then compute the likelihood of each data point belonging to each cluster $P(x^{(i)}|z)$.

0

We then update the centers and covariances of each cluster by only looking at the points that are more likely to belong to this cluster

0

Given the parameters of the learned Gaussian mixtures, we can update the clustering assignment based on Bayes rule $P(z|x) = P(x|z)P(z)/P(x)$

0

The loss function of the mixture model is non-convex and it has many local maxima, making optimization hard.

0

Start the presentation to see live content. For screen share software, share the entire screen. Get help at **pollev.com/app**

Expectation Maximization (EM): Intuition

$$\max_{\theta} \frac{1}{n} \sum_{i=1}^{n} \log\left(\sum_{z \in \mathcal{Z}} P_\theta(x^{(i)}, z) \right)$$

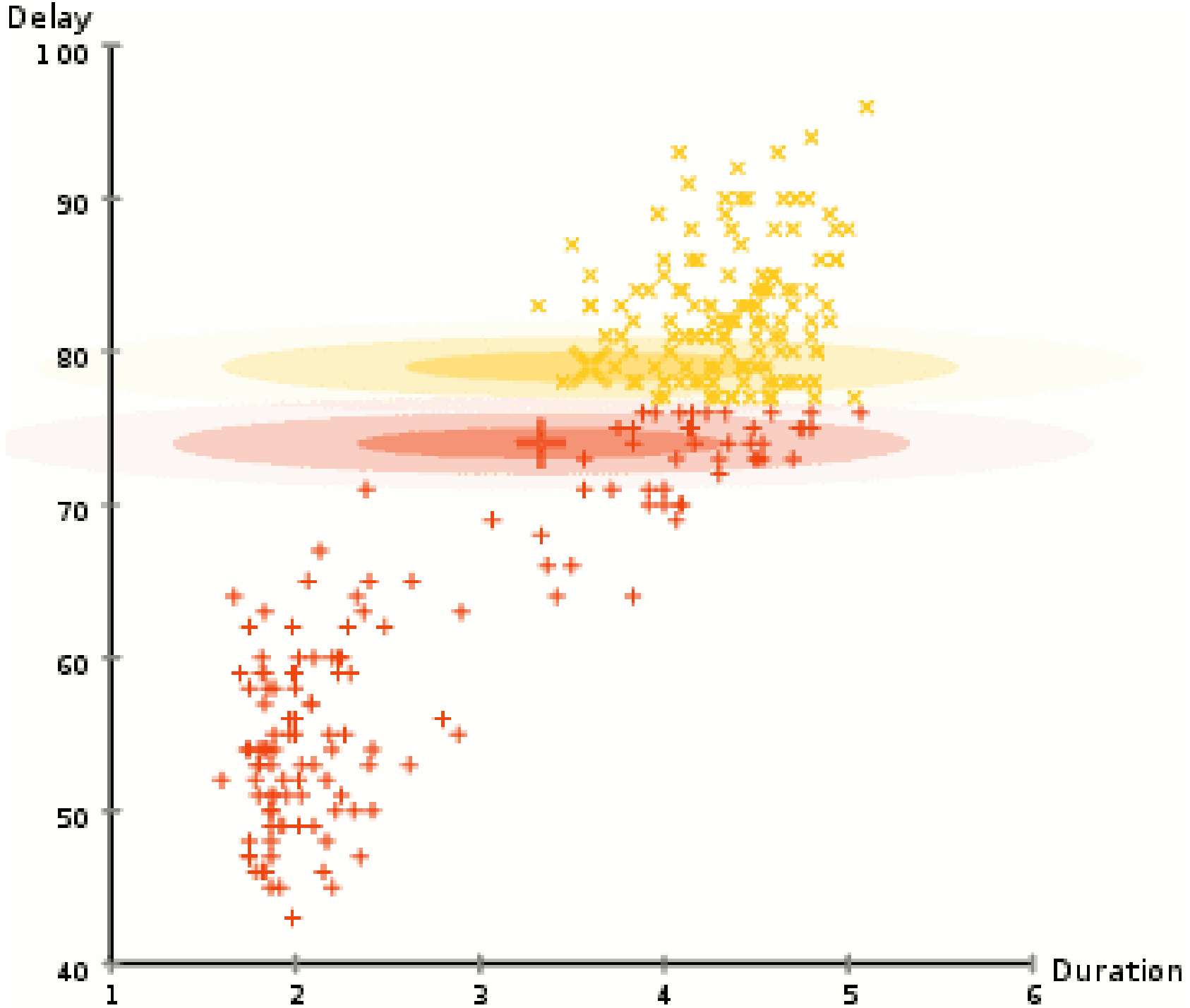
- If we know the true $z^{(i)}$ for each $x^{(i)}$, we maximize

$$\max_{\theta} \sum_{x^{(i)}, z^{(i)} \in \mathcal{D}} \log\left(P_\theta(x^{(i)}, z^{(i)}) \right).$$

- If we know θ, we can estimate the cluster assignments $z^{(i)}$ for each i by computing $P_\theta(z|x^{(i)})$.

Expectation Maximization (EM):

- Randomly initialize θ
- Repeat until convergence:
  - (**E-step**): Given the current estimate θ_t compute $P_\theta\left(z\middle|x^{(i)}\right)$. Use it to obtain cluster assignment $z^{(i)}$.
    - Similar to making a prediction GDA
  - (**M-step**): Find a new θ_{t+1} that maximizes the marginal log-likelihood by optimizing $P_\theta\left(\mathrm{z}^{(\mathrm{i})}, \mathrm{x}^{(i)}\right)$, given the $z^{(i)}$ from E-step.
    - Similar to calculating the mean and covariance for a fixed cluster assignment


Delay
100
90
80
70
60
50
40
1
2
3
4
5
6
Duration

E-Step

- Given the learned/initialized parameters $\boldsymbol{\phi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}$
- Assign cluster $z^{(i)}$ in the E-step
- $P_{\theta_t}\left(z|x^{(i)}\right)$ defined "soft" assignment
  - Compute a vector of class probabilities for each $x^{(i)}$

$$p\left(z^{(i)} = j|x^{(i)}; \phi, \mu, \Sigma\right) =$$

$$\frac{p\left(x^{(i)}|z^{(i)} = j; \mu, \Sigma\right) p\left(z^{(i)} = j; \phi\right)}{\sum_{l=1}^{k} p\left(x^{(i)}|z^{(i)} = l; \mu, \Sigma\right) p\left(z^{(i)} = l; \phi\right)}$$

M-Step

- Given the cluster assignment probabilities $P_{\theta_t}\left(z|x^{(i)}\right)$, we update the model parameters $\boldsymbol{\phi}, \boldsymbol{\mu}, \boldsymbol{\Sigma}$ that maximizes the expected likelihood of the data under this assignment

$$\theta_{t+1} = \arg\max_{\theta} \sum_{i=1}^{n} \mathbb{E}_{z^{(i)} \sim P_{\theta_t}(z|x^{(i)})} \log P_{\theta}(x^{(i)}, z^{(i)})$$

$$= \arg\max_{\theta} \sum_{i=1}^{n} \sum_{k=1}^{K} P_{\theta_t}(z = k|x^{(i)}) \log P_{\theta}(x^{(i)}, z = k)$$

our current inaccurate guess of the cluster likelihood

- Trackable for many interesting models